Symmetric distribution

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# Parameters of the symmetric distribution (Normal distribution)
mu = 0       # mean (center of the curve)
sigma = 1    # standard deviation

# Generate x values
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 500)

# Calculate y values (PDF of normal distribution)
y = norm.pdf(x, mu, sigma)

# Plot
plt.figure(figsize=(8, 5))
plt.plot(x, y)
plt.title("Symmetric Normal Distribution Curve")
plt.xlabel("x")
plt.ylabel("Probability Density")
plt.grid(True)

plt.show()
```

2)Right skewed distribution

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import skewnorm

# Right skew (positive skewness)
a = 6   # positive value → right skew

x = np.linspace(-5, 5, 500)
y = skewnorm.pdf(x, a)

plt.figure(figsize=(8, 4))
plt.plot(x, y)
plt.title("Right-Skewed Distribution (Positive Skew)")
plt.xlabel("x")
plt.ylabel("Probability Density")
plt.grid(True)
plt.show()
```

3)left skewed distribution

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import skewnorm

# Left skew (negative skewness)
a = -6   # negative value → left skew
```

```
x = np.linspace(-5, 5, 500)
y = skewnorm.pdf(x, a)

plt.figure(figsize=(8, 4))
plt.plot(x, y)
plt.title("Left-Skewed Distribution (Negative Skew)")
plt.xlabel("x")
plt.ylabel("Probability Density")
plt.grid(True)
plt.show()
```

4)kurtosis comparison

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import kurtosis

# ----------------------------
# Generate three distributions
# ----------------------------

# 1. Mesokurtic (Normal distribution)
data_normal = np.random.normal(loc=0, scale=1, size=1000)

# 2. Leptokurtic (Heavy tails) – Laplace distribution
data_lepto = np.random.laplace(loc=0, scale=1, size=1000)
```

```
# 3. Platykurtic (Light tails) – Uniform distribution
data_platy = np.random.uniform(low=-3, high=3, size=1000)

# ----------------------------
# Calculate kurtosis
# ----------------------------
k_normal = kurtosis(data_normal, fisher=True)
k_lepto  = kurtosis(data_lepto, fisher=True)
k_platy  = kurtosis(data_platy, fisher=True)

print("Kurtosis Values (Fisher=True):")
print(f"Mesokurtic (Normal)     : {k_normal:.3f}")
print(f"Leptokurtic (Laplace)   : {k_lepto:.3f}")
print(f"Platykurtic (Uniform)   : {k_platy:.3f}")

# ----------------------------
# Plot all three distributions
# ----------------------------
plt.figure(figsize=(10,6))

plt.hist(data_normal, bins=30, density=True, alpha=0.5, label="Normal (Mesokurtic)")
plt.hist(data_lepto, bins=30, density=True, alpha=0.5, label="Laplace (Leptokurtic)")
plt.hist(data_platy, bins=30, density=True, alpha=0.5, label="Uniform (Platykurtic)")

plt.title("Kurtosis Comparison: Normal vs Leptokurtic vs Platykurtic")
```

```
plt.xlabel("Value")
plt.ylabel("Density")
plt.legend()
plt.show()
```

5)quantile calculation

```
import numpy as np

data = [12, 15, 18, 21, 24, 27, 30, 33, 36]

# Calculate quantiles
q1 = np.quantile(data, 0.25)   # 25th percentile
q2 = np.quantile(data, 0.50)   # 50th percentile (median)
q3 = np.quantile(data, 0.75)   # 75th percentile
q90 = np.quantile(data, 0.90)  # 90th percentile

print("Q1 (25%):", q1)
print("Q2 (Median):", q2)
print("Q3 (75%):", q3)
print("90th Percentile:", q90)
```

6)box plot showing quantiles

```
import numpy as np
import matplotlib.pyplot as plt

# Generate random data
data = np.random.normal(loc=50, scale=12, size=500)
```

```
plt.figure(figsize=(7, 5))
plt.boxplot(data, vert=True)

plt.title("Box Plot Showing Q1, Median, and Q3")
plt.ylabel("Value")
plt.grid(True)

plt.show()
```

7)quantile curve

```
import numpy as np
import matplotlib.pyplot as plt

# Generate random data
data = np.random.normal(loc=40, scale=15, size=1000)

# Calculate percentiles for every 1%
percentiles = np.arange(0, 101, 1)
values = np.percentile(data, percentiles)

plt.figure(figsize=(8, 5))
plt.plot(percentiles, values)

plt.title("Quantile (Percentile) Curve")
plt.xlabel("Percentile")
plt.ylabel("Value")
```

```
plt.grid(True)

plt.show()
```

8) compare quantile of two datasets

```
import numpy as np
import matplotlib.pyplot as plt

# --- Create two different datasets ---
np.random.seed(42)

# Dataset A: Normal (symmetric)
data_A = np.random.normal(loc=50, scale=10, size=1000)

# Dataset B: Right-skewed (exponential + shift)
data_B = np.random.exponential(scale=20, size=1000) + 30

# --- Calculate percentiles (0% to 100%) ---
percentiles = np.arange(0, 101, 1)
quantiles_A = np.percentile(data_A, percentiles)
quantiles_B = np.percentile(data_B, percentiles)

# --- Plot quantile curves ---
plt.figure(figsize=(9, 6))
plt.plot(percentiles, quantiles_A, label="Dataset A (Normal)", linewidth=2)
plt.plot(percentiles, quantiles_B, label="Dataset B (Right-Skewed)", linewidth=2)
```

```
plt.title("Comparison of Quantile Curves for Two Datasets")
plt.xlabel("Percentile")
plt.ylabel("Value")
plt.grid(True)
plt.legend()

plt.show()
```

9) Central Limit Theorem — Python Program

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate a non-normal population (right-skewed)
population = np.random.exponential(scale=2, size=100000)

# Step 2: Function to draw sample means
def sample_means(sample_size, num_samples):
    means = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=sample_size, replace=True)
        means.append(np.mean(sample))
    return np.array(means)

# Step 3: Generate sample means for different sample sizes
means_n5  = sample_means(5, 1000)
means_n30 = sample_means(30, 1000)
```

```
means_n100 = sample_means(100, 1000)

# Step 4: Plot to show CLT effect
plt.figure(figsize=(12, 8))

plt.subplot(3, 1, 1)
plt.hist(means_n5, bins=30)
plt.title("Sample Means (n=5)   — Slightly Normal")

plt.subplot(3, 1, 2)
plt.hist(means_n30, bins=30)
plt.title("Sample Means (n=30)  — More Normal")

plt.subplot(3, 1, 3)
plt.hist(means_n100, bins=30)
plt.title("Sample Means (n=100) — Very Normal (CLT)")

plt.tight_layout()
plt.show()
```

10) **Python Program: Demonstrating Common Population Parameters**

```
import numpy as np
from scipy.stats import skew, kurtosis

# ----------------------------------------------------------
# 1. Create a population (example: right-skewed exponential)
```

```
# ------------------------------------------------------------

population = np.random.exponential(scale=10, size=50000)

# ------------------------------------------------------------

# 2. Calculate common population parameters

# ------------------------------------------------------------

# Mean (μ)

pop_mean = np.mean(population)

# Variance (σ²)

pop_variance = np.var(population)  # population variance (ddof=0)

# Standard Deviation (σ)

pop_std = np.std(population)

# Skewness

pop_skew = skew(population, bias=False)

# Kurtosis (Fisher's definition: normal => 0)

pop_kurtosis = kurtosis(population, fisher=True, bias=False)

# Quantiles

q1 = np.quantile(population, 0.25)

q2 = np.quantile(population, 0.50)

q3 = np.quantile(population, 0.75)
```

```
# ----------------------------------------------------------
# 3. Print results
# ----------------------------------------------------------
print("=== Population Parameters ===")
print("Population Mean (μ):", round(pop_mean, 3))
print("Population Variance (σ²):", round(pop_variance, 3))
print("Population Std Dev (σ):", round(pop_std, 3))
print("Population Skewness:", round(pop_skew, 3))
print("Population Kurtosis:", round(pop_kurtosis, 3))
print("Q1 (25th percentile):", round(q1, 3))
print("Median (Q2):", round(q2, 3))
print("Q3 (75th percentile):", round(q3, 3))
```

11) **Python Program: Common Sample Statistics**

```
import numpy as np
from scipy.stats import skew, kurtosis

# ----------------------------------------------------------
# 1. Create a population (for demonstration)
# ----------------------------------------------------------
population = np.random.exponential(scale=10, size=50000)

# ----------------------------------------------------------
# 2. Draw a sample from the population
```

```
# ------------------------------------------------------------

sample = np.random.choice(population, size=100, replace=False)

# ------------------------------------------------------------

# 3. Calculate common sample statistics

# ------------------------------------------------------------

# Sample mean

sample_mean = np.mean(sample)

# Sample variance (unbiased → ddof=1)

sample_variance = np.var(sample, ddof=1)

# Sample standard deviation

sample_std = np.std(sample, ddof=1)

# Sample skewness

sample_skew = skew(sample, bias=False)

# Sample kurtosis (Fisher => normal = 0)

sample_kurtosis = kurtosis(sample, fisher=True, bias=False)

# Quartiles

q1 = np.quantile(sample, 0.25)

q2 = np.quantile(sample, 0.50)

q3 = np.quantile(sample, 0.75)
```

```
# Standard Error of the Mean
sem = sample_std / np.sqrt(len(sample))

# ------------------------------------------------------------
# 4. Print the results
# ------------------------------------------------------------
print("=== Sample Statistics ===")
print("Sample Mean (x̄):", round(sample_mean, 3))
print("Sample Variance (s²):", round(sample_variance, 3))
print("Sample Std Dev (s):", round(sample_std, 3))
print("Sample Skewness:", round(sample_skew, 3))
print("Sample Kurtosis:", round(sample_kurtosis, 3))
print("Q1 (25%):", round(q1, 3))
print("Median (Q2):", round(q2, 3))
print("Q3 (75%):", round(q3, 3))
print("Standard Error of Mean (SEM):", round(sem, 3))
```

12)program to calculate standard error

```
import numpy as np

# Sample data (you can replace this list with your own values)
sample = [12, 15, 18, 20, 22, 25, 30, 28, 26, 24]

# Step 1: Calculate sample standard deviation (ddof=1 for unbiased estimate)
sample_std = np.std(sample, ddof=1)
```

```
# Step 2: Sample size
n = len(sample)

# Step 3: Standard Error
standard_error = sample_std / np.sqrt(n)

print("Sample Standard Deviation:", round(sample_std, 3))
print("Standard Error (SE):", round(standard_error, 3))
```

13)standard error for randomly generated sample

```
import numpy as np

# Generate random sample
sample = np.random.normal(loc=50, scale=10, size=100)

sample_std = np.std(sample, ddof=1)
n = len(sample)
se = sample_std / np.sqrt(n)

print("Standard Error:", round(se, 4))
```